MarketsUI Platform — Feature Inventory (Reconciled)

What actually ships today, mapped to the module that implements it

2026-05-17

Contents

About this document

This file is the reconciled, code-first feature inventory for the MarketsUI / StarUI monorepo. It supersedes the chronological changelog in `IMPLEMENTED_FEATURES.md` for the question **“what does the platform actually do, and where does each feature live?”**

The changelog stays useful as historical record, but it has accumulated stale references after several refactors (notably the PR-8 module extraction from `@starui/core` into `@starui/grid-react`, the `--ds-*` design-token sweep, the v2 data-plane rewrite, the WorkspaceSetup consolidation, and the workspace-bucket reorg under `packages/{shared,react,angular}/`). Every feature listed below was confirmed by reading source files under `/Users/develop/wfh/starui/` on 2026-05-17.

Conventions:

- A *feature* is described from the user’s / consumer’s point of view (what the platform does), and is paired with the **package and file path** that owns it today.
- Each section is prefixed with the package that hosts the feature, so a developer reading the inventory can jump straight to the source.

- Sections marked **[stale-in-changelog]** flag features the older doc still describes in terms of paths that no longer exist (e.g. `packages/core-v2/` or `packages/markets-grid-v2/`). The behaviour still ships; only the location has moved.

The full reconciliation rollup at the end (§14) lists every notable discrepancy between `IMPLEMENTED_FEATURES.md` and the live codebase.

---

1. Monorepo at a glance

29 published packages + 6 apps. Layout under `packages/`:

```
packages/
├── shared/                framework-agnostic vanilla TS (12)
│    ├── foundation/          shared-types · design-system · icons-svg · vite-workspace-aliases
│    ├── runtime/            runtime-port · runtime-browser · runtime-openfin
│    ├── services/           config-service · data-services · component-host
│    ├── platform/           openfin-platform
│    └── core               GridPlatform, ProfileManager, ExpressionEngine, modules
│
├── react/                 React-only (12)
│    ├── ui                  shadcn primitives (singleton — no `-react` suffix)
│    ├── sdk/widget-sdk      widget contract
│    ├── widgets/            grid-react · markets-grid · widgets-react
│    ├── hosts/              host-wrapper-react · app-shell-react
│    ├── providers/          config-service-react · data-services-react
│    └── tools/               config-browser-react · config-editor-ui · workspace-setup-react
│
└── angular/               Angular-only (5; parity catching up)
     ├── hosts/            host-wrapper-angular
     ├── providers/        config-service-angular · data-services-angular
     ├── tools/            config-browser-angular
     └── widgets/          widgets-angular
```

Apps under `apps/`:

App	Stack	Role
`demo-react`	React 19 + Vite 7	Primary E2E target. Single-grid blotter with view switcher (single / dashboard / depth / design-system / stockflux-blotter / fixture).
`demo-angular`	Angular 21 + PrimeNG 21	Angular dock-manager demo with 32 widget components and trading layouts.
`demo-configservice-react`	React 19	ConfigService walkthrough — multi-user picker (dev1/alice/bob), ConfigBrowser popout via window.open.

App	Stack	Role
markets-ui-react-reference	React 19 + OpenFin Workspace	The reference OpenFin shell: provider window, blotter, data-providers admin, config-browser, workspace-setup, rename-view-tab popout.
demo-apps/basic-starui-app	React 19	Minimal “just MarketsGrid” example using createMarketsGridLocal-StorageStorage() and the bundle adapter.
stomp-view-server	Node 18 + ws	STOMP-over-WebSocket fixed-income data server used by demo blotters.

apps/fi-trading-reference, apps/fi-trading-reference-angular, and apps/markets-ui-angular-reference listed in the older README do not exist in the working tree — they were removed during the consolidation work.

---

2. Layer 0 — Foundation

2.1 @starui/shared-types (packages/shared/foundation/shared-types/)

The platform’s typing root. Imports nothing.

- **Configuration row contract**: AppConfigRow, UnifiedConfig, ConfigurationFilter, PaginatedResult<T>, StorageHealthStatus, BulkUpdate*, CleanupResult, the COMPONENT_TYPES / COMPONENT_SUBTYPES catalogue (dock-config, component-registry, markets-grid-profile-set, etc.) with derived union types.
- **Data-provider contract**: PROVIDER_TYPES (mock, stomp, rest, websocket, socketio, appdata), CONNECTION_STATES, FieldInfo, ColumnDefinition, per-transport configs (StompProviderConfig, RestProviderConfig, WebSocketProviderConfig, SocketIO-ProviderConfig, MockProviderConfig, AppDataProviderConfig, AppDataVariable) plus the umbrella ProviderConfig union, ProviderCapabilities, ProviderStatistics, validation results, composite-key helpers (normalizeKeyColumns, getValueByPath, composeRowId, COMPOSITE_KEY_SEPARATOR).
- **SimpleBlotter contract**: toolbars (ToolbarZone, ToolbarButton, DynamicToolbar, BUILT_IN_ACTIONS), conditional formatting types (ConditionalFormatStyle, ConditionalFormatRule), editing rules (EditingCondition, EditingValidation, EditingRule), value formatters (FormatterType, FormatterOptions, ValueFormatterDef), calculated columns (CalculatedColumnDef), grid state shapes, SimpleBlotterConfig, factories createDefaultBlotterConfig / createDefaultLayoutConfig.
- **Dock contract**: DockMenuItem, DockButton, DockButtonOption, DEFAULT_WINDOW_OPTIONS, DEFAULT_VIEW_OPTIONS, plus tree utilities (findMenuItem, updateMenuItem, deleteMenuItem, addMenuItem, duplicateMenuItem, moveMenuItem, countItems, getAllItemIds).
- **Field selector**: FieldNode, conversion helpers, leaf collection, path lookup, filter.
- **Widget routing**: WidgetRouteEntry, LayoutInfo.

2.2 @starui/design-system (packages/shared/foundation/design-system/)

Single source of truth for visual design across React, Angular, AG-Grid, and PrimeNG.

- **Token tree** (src/tokens/): primitives, semantic.dark / semantic.light / semantic.shared, componentTokens, controls (with ControlSize / ControlTier). Stockflux-slate palette source.
- **Framework adapters** (src/adapters/):
  - tailwindPreset
  - primengPreset
  - generateUnifiedCSS (shadcn CSS-vars generator)
  - AG-Grid theme bakers: agGridDarkParams, agGridLightParams, plus comfort and blotter variants (agGridComfortDarkParams, agGridComfortLightParams, agGridBlotterDarkParams, agGridBlotterLightParams) and pre-baked theme objects.
- **applyTheme() / getTheme()** — writes / reads [data-theme] and persists to localStorage.
- **Cell renderers** (src/cellRenderers.ts): 13 framework-agnostic ICellRendererComp implementations — SideCellRenderer, StatusBadgeRenderer, ColoredValueRenderer, OasValueRenderer, SignedValueRenderer, TickerCellRenderer, RatingBadgeRenderer, PnlValueRenderer, FilledAmountRenderer, BookNameRenderer, ChangeValueRenderer, YtdValueRenderer, RfqStatusRenderer.
- **Bundled stylesheet** (./css subpath): built into dist/css/theme.css from src/styles/{base,scrollba plus src/themes/{fi-dark,fi-light,scrollbars}.css.
- **WCAG contrast audit** (src/internal/wcag.ts) and the check-ds-tokens / check-design-system-deps / check-react-apps-no-native-select scripts under tools/scripts/.
- **Subpaths**: ., ./css, ./tailwind, ./primeng, ./shadcn, ./adapters/ag-grid, ./tokens, ./tokens/primitives, ./tokens/semantic, ./tokens/components, ./tokens/controls, ./cell-renderers.

2.3 @starui/icons-svg (packages/shared/foundation/icons-svg/)

- ~125 SVG icons in svg/*.svg using currentColor, categorised (trading, blotters, charts, risk, general, system).
- Public API: ICON_PATHS, MarketIconName, ICON_META, ICON_NAMES, ICON_CATEGORIES, ICON_CATEGORY_NAMES, getIconsByCategory.
- MARKET_ICON_SVGS — raw SVG strings consumed by the dock editor icon pickers.
- **React binding** (./react): curated lucide-react re-exports plus DynamicIcon (string-id → component).
- **Angular binding** (./angular): parallel surface.
- Subpaths: ., ./react, ./angular, ./all-icons, ./svg/*.

2.4 @starui/ui (packages/react/ui/)

The shadcn/Radix primitive library for the platform. Themed via @starui/design-system.

- **Layout / containers**: Accordion, AspectRatio, Card, Collapsible, Resizable, ScrollArea, Separator, Sheet, Tabs.
- **Navigation**: Breadcrumb, DropdownMenu, Menubar, NavigationMenu, Pagination, ContextMenu, Command (cmdk).
- **Forms / inputs**: Button, ButtonGroup, Checkbox, Form (react-hook-form), Input, InputOTP, Label, RadioGroup, Select, Slider, Switch, Textarea, Toggle, ToggleGroup.
- **Data display**: Avatar, Badge, Calendar (react-day-picker), Carousel (embla), Progress, Skeleton, Table.
- **Overlays / feedback**: Alert, AlertDialog, Dialog, Drawer (vaul), HoverCard, Popover, Sonner, legacy Toast / Toaster, Tooltip, useToast.
- **Trading-platform extras**: CollapsibleToolbar, ToolbarContainer, VirtualizedList.

- **Theme + portal infrastructure**: `ThemeProvider` + `useTheme`, `PortalContainerProvider`, `usePortalContainer`, `useResolvedPortalContainer`. Portal targeting threads through every Radix primitive so popouts and OpenFin child windows mount overlays into their own `document.body`.
- **Chart** (`./chart` subpath) — Recharts wrapper kept off the default barrel so consumers that don't render charts skip the bundle cost.

2.5 `@starui/vite-workspace-aliases`

A single helper `buildPackageAliases(opts)` that walks the npm workspaces manifest and emits Vite `resolve.alias` entries mapping each `@starui/*` export to its `src/` file. Eliminates per-app `vite.config.ts` drift.

---

3. Layer 1 — Runtime

3.1 `@starui/runtime-port` (`packages/shared/runtime/runtime-port/`)

Pure interface package (no peer deps). Exports:

- **`RuntimePort`** — `name`, `resolveIdentity`, `openSurface`, `getTheme`, `setTheme`, `onThemeChanged`, `onWindowShown`, `onWindowClosing`, `onCustomDataChanged`, `onWorkspaceSave`, `dispose`.
- **Types**: `IdentitySnapshot` (instanceId / appId / userId / componentType / componentSubType / isTemplate / singleton / roles / permissions / customData), `Theme`, `SurfaceKind` (`popout` / `modal` / `inpage`), `SurfaceSpec`, `SurfaceHandle`, `Unsubscribe`.
- **Constants**: `LOGGED_IN_USER_ID = 'dev1'` (single-user pin until SSO lands), `THEME_STORAGE_KEY = 'starui:theme'`, `THEME_BROADCAST_CHANNEL = 'starui:theme'`.

3.2 `@starui/runtime-browser`

- `BrowserRuntime` implementing `RuntimePort`. Identity comes from URL params; theme from `prefers-color-scheme` + persisted localStorage + `[data-theme]`; cross-tab sync via `BroadcastChannel('starui:theme')`.
- Popouts use `window.open`; in-page modal is implemented for `kind: 'modal'`.
- `resolveBrowserIdentity(IdentityOverrides)` for non-runtime callers.

3.3 `@starui/runtime-openfin`

- `OpenFinRuntime` — `fin.*` wrapper. Reads identity from view `customData` with URL / mount-prop fallbacks; bridges `theme-changed`, `workspace-saved`, `customData`, `window-shown`, `close-requested`.
- `applySavedViewTitle()` — re-applies `customData.savedTitle` to `document.title` on view boot, with a 3-second `MutationObserver` guard that defeats the app's own `useEffect`-mounted `document.title = ...` so the "Save Tab As..." rename survives workspace restore.
- `resolveOpenFinIdentity(OpenFinIdentitySources)`, `isOpenFin()`, `getCurrentView()`, `openOpenFinPopout` exported alongside.
- `userId` is hard-pinned to `LOGGED_IN_USER_ID = 'dev1'` regardless of what `customData.userId` carries — single-user contract until SSO.

---

4. Layer 2 — Services & platform

4.1 `@starui/core` (`packages/shared/core/`)

The framework-agnostic heart of the grid platform. **Note for changelog readers:** after PR-8, all grid modules and React UI primitives moved out of core into `@starui/grid-react`. Today, core is *vanilla TS only*.

- **Platform runtime** (`src/platform/`): `GridPlatform` (`onGridReady`, `destroy`, `transformColumnDefs`, `transformGridOptions`, `serializeAll`, `deserializeAll`, `resetAll`, `getModules`), supporting machinery — `EventBus`, `topoSortModules`, `ApiHub`, `ResourceScope`, `CssInjector`, `DirtyBus`, `PipelineRunner`. Types: `Module`, `AnyModule`, `AnyColDef`, `ApiEventName`, `AppDataLookup`, `CssHandle`, `EditorPaneProps`, `ExpressionEngineLike`, `GridApi`, `GridOptions`, `GetRowIdFunc/Params`, `ListPaneProps`, `PlatformEventMap`, `PlatformHandle`, `SerializedState`, `SettingsPanelProps`, `Store`, `TransformContext`.
- **Store + autosave** (`src/store/`): `createGridStore` (Zustand factory), `startAutoSave` (`AutoSaveOptions`, `AutoSaveHandle`).
- **Persistence adapters** (`src/persistence/`): `StorageAdapter` interface, `MemoryAdapter`, `LocalStorageBundleAdapter` (with `marketsGridLocalStorageBundleKey`, `MarketsGridLocalStorageConfig`). Dexie- and ConfigService-backed adapters live in `@starui/config-service`, not core.
- **ProfileManager** (`src/profiles/`): `boot`, `save`, `discard`, `load`, `create`, `remove`, `rename`, `clone`, `export`, `import`, `subscribe`. Types: `ActiveIdSource` (pluggable per-view active-id source — OpenFin reads/writes `customData.activeProfileId`), `ProfileMeta`, `ExportedProfilePayload`.

  Performance note: `load()` no longer calls `refresh()` at the end (the `listProfiles` round-trip was pure overhead) — ~5× wall-clock cut on localStorage hosts.
- **ExpressionEngine** (`src/expression/`): full lexer / parser / evaluator with 44 built-in functions across 6 categories:
  - **Math** (11): `ABS`, `ROUND`, `FLOOR`, `CEIL`, `SQRT`, `POW`, `MOD`, `LOG`, `EXP`, `MIN`, `MAX`
  - **Stats** (4): `AVG`, `MEDIAN`, `STDEV`, `VARIANCE`
  - **Aggregation** (3): `SUM`, `COUNT`, `DISTINCT_COUNT`
  - **String** (11): `CONCAT`, `UPPER`, `LOWER`, `TRIM`, `SUBSTRING`, `REPLACE`, `LEN`, `STARTS_WITH`, `ENDS_WITH`, `CONTAINS`, `REGEX_MATCH`
  - **Date** (8): `NOW`, `TODAY`, `YEAR`, `MONTH`, `DAY`, `IS_WEEKDAY`, `DATE_DIFF`, `DATE_ADD`
  - **Logical** (7): `IF`, `IFS`, `SWITCH`, `CASE`, `ISNULL`, `ISNOTNULL`, `ISEMPTY`

  Column-aware aggregation: 9 functions (`SUM`, `COUNT`, `DISTINCT_COUNT`, `AVG`, `MEDIAN`, `STDEV`, `VARIANCE`, `MIN`, `MAX`) carry `aggregateColumnRefs: true` so direct `[col]` args are replaced with the full column array before the function runs.

  Helpers: `migrateExpressionSyntax`, `migrateExpressionsInObject`, `tryCompileToAgString` (for predicate compilation into AG-Grid filter primitives).
- **Security** (`src/security/`): `configureExpressionPolicy({ mode, onViolation })` and `getExpressionPolicy()` — runtime switch (`allow` / `warn` / `strict`) gating the `kind: 'expression'` ValueFormatterTemplate path (compiled via `new Function`). Strict-mode import scan with optional `{ sanitize: true }` in-place rewrite.
- **History** (`src/history/`): `HistoryStack` framework-agnostic undo / redo core.
- **colDef helpers** (`src/colDef/`): `BorderSpec`, `CellStyleOverrides`, `ColumnAssignment`, `ColumnDataType`, `GridThemeMode`, `PresetId`, `ThemedCellStyleOverrides`, `TickToken`, `ValueFormatterTemplate`, themed-style helpers (`getActiveTheme`, `resolveActiveStyle`, `patchActiveStyle`, `mergeThemedStyle`, `migrateThemedStyle`), adapters

(cellStyleToAgStyle, valueFormatterFromTemplate, excelFormatter + excelFormatColorResolver + isValidExcelFormat, tickFormatter, presetToExcelFormat).

- **CSS** (src/css/): injectEditorStyles (idempotent, SSR-safe).
- **OpenFin shim** (src/utils/openFin.ts): isOpenFin, openFinWindowOpener.

4.2 @starui/config-service (packages/shared/services/config-service/)

Dual-mode configuration service: Dexie / IndexedDB in local-dev mode, REST + Dexie in production mode.

- **ConfigClient** (preferred surface for new code): framework-agnostic REST-shaped interface, with three concrete implementations — LocalConfigClient (Dexie-backed), RestConfigClient (HTTP), and a createConfigClient dispatcher. Error types ConfigNotFoundError, ConfigClientHttpError, OptimisticLockError.
- **ConfigManager** (createConfigManager): owns the live Dexie database. Tested for audit stamping, identity / impersonation (ImpersonatedUser, getEffectiveUser), optimistic locking, visibility filter (isVisible(VisibilityContext)), profiles namespace, application-context publishing.
- **AppDataMirror integration**: publishApplicationContext() does a single publishNamedRow upsert into AppData (replaces the older four-set round-trip on the SharedWorker).
- **createConfigServiceStorage({ configManager, appId, userId })** — the StorageAdapterFactory that backs MarketsGrid's storage prop. Persists one AppConfigRow per instance (componentType: "markets-grid-profile-set") with all profiles bundled in the payload. Brand CONFIG_SERVICE_ADAPTER_BRAND, helper getConfigServiceAdapterBrand, types ConfigServiceStorageOptions, ProfileStorageFactory, ProfileStorageFactoryOpts, RegisteredComponentIdentity.
- **ConfigManager.profiles namespace** — first-class read/write over bundled profile-set rows (ProfilesNamespace, ProfilesScope, ProfilesSaveOptions).
- **migrateProfilesToConfigService({ source, target, gridId })** — one-shot migration helper from DexieAdapter / MemoryAdapter → ConfigService storage.
- **migrateLegacyProfilesIfNeeded()** — boot-time consolidation from the legacy gc-customizer-v2 Dexie database into the bundled row.
- **ChangeNotifier** — cross-tab + same-tab event bus backing profiles.subscribe.
- **ProfileSetVersionConflictError** — surfaced when a bundle write loses an optimistic-lock race.

4.3 @starui/data-services (packages/shared/services/data-services/)

SharedWorker-backed data-services runtime — one network connection per provider, many consumers multiplexed.

- **Bootstrap surface**: bootstrapDataServices() (idempotent by appName), createDataServicesClient (raw factory), DataServices (the bundled handle: client + AppDataMirror + ConfigManager + ready handle).
- **Protocol** (src/runtime/protocol.ts): typed request / event union for the worker bus — AttachRequest, DetachRequest, StopRequest, AppData ops (AppDataAttachRequest, AppDataDetachRequest, AppDataSetRequest, AppDataUpsertRequest, AppDataRemoveRequest); events DeltaEvent, StatusEvent, StatsEvent, AppDataSnapshotEvent, AppDataDeltaEvent, AppDataAckEvent. ProviderStats, ProviderStatus ('loading' | 'ready' | 'error').
- **SharedWorker client**: SharedWorkerDataServicesClient — attach({ subId, providerId, cfg?, mode, extra? }), detach, stop. DataListener, StatsListener, SubId, SubscribeHandle.
- **SharedWorker hub**: SharedWorkerDataServicesHub, WorkerAppDataStore, installSharedWorkerHub, plus entry.ts / defaultEntry.ts worker entry points.

- **Provider registry**: registerProvider, startProvider, ProviderFactory, ProviderEmit, ProviderHandle.
- **Transports** (src/runtime/providers/transports/):
  - **Mock** — startMock, probeMock, plus generators mockUniverse, mockPosition, mockTrade.
  - **STOMP** — startStomp, probeStomp, injectable StompClientFactory, StompProbeResult.
  - **REST** — startRest, probeRest, RestFetchFn.
- **Field inference**: inferFields(InferOptions) — completeness- weighted sampling returning a tree of FieldNode.
- **Template substitution** (src/runtime/template/):
  - {{name.key}} — runs on the main thread before attach, against AppDataLookup snapshot (resolveTemplate, resolveCfg, collectTemplateRefs).
  - [token] — BracketCache, resolveBracketString, resolveBracketCfg — mints a per-attach 12-char alphanumeric ID shared across every occurrence of the same token in one provider config. Re-attach mints fresh values.
- **Config stores**: DataProviderConfigStore (COMPONENT_TYPE_DATA_PROVIDER, PUBLIC_USER_ID, ListOptions), AppDataConfigStore (COMPONENT_TYPE_APPDATA).
- **AppDataMirror** — sync-read + async-write proxy over the worker store.
- **DataProviderConfigService** singleton at the package level (src/services/dataProviderConfigServic plus a DataProviderLocalBackend.

4.4 @starui/component-host

A lightweight component-hosting wrapper that injects OpenFin platform services (identity, config, theme) into registered components.

- Vanilla core (src/): readCustomData, resolveInstanceId(identity, configManager) (loads existing config or clones a template per MarketsUI design doc §6.4), buildFallbackIdentity, subscribeToTheme / getCurrentTheme (IAB-backed), onCloseRequested (workspace close hook), createDebouncedSaver (DebouncedSaver, DebouncedSaverOptions) with flush and cancel.
- **React** (./react): useComponentHost<T>() → { instanceId, config, theme, isLoading, isSaved, error, saveConfig(partial) }.
- **Angular** (./angular): ComponentHostService<T> — @Injectable(), per-component scope, exposes readonly signals, init(options?), saveConfig(partial), automatic ngOnDestroy cleanup.

4.5 @starui/openfin-platform (packages/shared/platform/openfin-platform/)

The OpenFin Workspace shell. Only this package and runtime-openfin may import from @openfin/core. Two subpaths:

- **.** — full surface (pulls @openfin/workspace-platform — OpenFin-only).
- **./config** — side-effect-free subset safe to import in plain browser tools (Config Browser, View routes outside OpenFin).

Features:

- **Workspace bootstrap**: initWorkspace(WorkspaceConfig?) — single entry that registers home / store / dock / notifications and the CustomActions map.
- **Launch helpers**: launchApp, launchRegisteredComponent, resolveHostUrl, openChildToolWindow, openDataProvidersToolWindow.
- **Dock management** (dock.ts): registerDock, updateDockButtons, getDefaultEditorConfig, recolorDockIcons, reloadDockFromConfig, shutdownDock. IAB topics:

IAB_DOCK_CONFIG_UPDATE, IAB_RELOAD_AFTER_IMPORT, IAB_THEME_CHANGED, IAB_REGISTRY_CONFIG_UPD
Action ids: ACTION_OPEN_REGISTRY_EDITOR, ACTION_OPEN_CONFIG_BROWSER, ACTION_LAUNCH_COMPONENT,
ACTION_LAUNCH_APP, ACTION_TOGGLE_THEME, ACTION_OPEN_DOCK_EDITOR, ACTION_RELOAD_DOCK,
ACTION_SHOW_DEVTOOLS, ACTION_EXPORT_CONFIG, ACTION_IMPORT_CONFIG, ACTION_TOGGLE_PROVIDER.

- **View-tab rename** ("Save Tab As..."): internal/viewTabRename.ts exports ACTION_RENAME_VIEW_TAB, RENAME_VIEW_TAB_WINDOW_NAME, injectRenameMenuItem, createRenameViewTabAction. Companion popout lives at apps/markets-ui-react-reference/src/views/RenameViewTab.tsx.
- **ConfigManager singleton + persistence** (db.ts): getConfigManager, setConfigManager, setPlatformDefaultScope, getPlatformDefaultScope, migrateLegacyPlatformScope, realignAllConfigsToPlatformScope, migrateRegistryToGlobalScope, plus typed save/load/clear for dock + registry configs (saveDockConfig, loadDockConfig, clearDockConfig, the matching Registry trio). ConfigScope and scopedConfigId for per-(appId, userId) rows.
- **Config import** (configImport.ts): importConfigBundle() ingests full export bundles (appConfig + appRegistry + roles + permissions), reowning each row to local (appId, userId). Sentinel values userId === 'system' and appId === '' are preserved; userProfile rows are intentionally NOT imported (userId is their primary key). Modes: overwrite, skip-existing.
- **Registry v2 schema & migrator** (registryConfigTypes.ts, registryValidate.ts, registryMigrate.ts): deriveTemplateConfigId, deriveSingletonConfigId, mintRegisteredInstanceId(userId, componentType, componentSubType) — produces deterministic, scan-friendly ids like dev1blotter-markets-1714999999999 (per §1.U of the changelog).
- **Manifest-driven config** (manifestConfig.ts — ./config subpath): resolveRestUrl, getConfigServiceRestUrlFromManifest — read REST vs local mode from manifest.fin.json customSettings.
- **Notifications / home / store** entry stubs (notifications.ts, home.ts, store.ts).
- **Custom actions** (internal/customActions.ts) wiring app-level menu / button / dropdown / view-tab-context-menu actions into the WorkspacePlatformOverrideCallback.
- **Icon library re-export** (icons/index.ts): MARKET_ICON_SVGS, svgToDataUrl, marketIconToDataUrl.

---

5. Layer 3 — Hosts, providers, SDK

5.1 @starui/widget-sdk (packages/react/sdk/widget-sdk/)

The Widget SDK. Used by widget-style hosts (not by MarketsGrid directly, which has its own contract).

- Types: WidgetConfig, WidgetProps, WidgetContext, WidgetHostProps, PlatformAdapter, ParentIdentity, slot types (SlotContent, WidgetEnhancer, ActionContext, WidgetExtensionConfig), SettingsScreenContext / Definition.
- WidgetRegistry class mapping widget type strings → React components.
- WidgetHost provider + useWidgetHost() hook (TanStack QueryClient, apiUrl, userId, platform, registry, configClient).
- Hooks: useWidget, useSettingsScreen.
- BrowserAdapter implementing PlatformAdapter (window.open, BroadcastChannel, crypto.randomUUID, save / destroy / settings-result handlers).
- Layout helpers around ConfigClient: getLayouts, saveLayout, loadLayout, deleteLayout.
- Extensibility primitives: renderSlot, createExtendedWidget, compose.

5.2 @starui/host-wrapper-react

Architecture Seam #2 for React. One component, one hook, one context.

- <HostWrapper runtime configManager configUrl? loading?> — awaits promise-shaped runtime / configManager before rendering.
- HostContext (HostContextValue extends IdentitySnapshot) — exposes runtime, configManager, theme, configUrl, setTheme(theme), onThemeChanged, onWindowShown, onWindowClosing, onCustomDataChanged, onWorkspaceSave.
- useHost() — throws if used outside <HostWrapper>.
- ./test-bridge subpath — dev-only installTestBridge plumbing (gated on import.meta.env.DEV).

5.3 @starui/host-wrapper-angular

Angular twin (full parity with useHost()):

- provideHostWrapper({ runtime, configManager, configUrl? }) provider factory.
- HostService (@Injectable({ providedIn: 'root' })) — exposes identity (instanceId, appId, userId, componentType, ...), runtime, configManager, configUrl?, themeSignal (Signal) + theme$ (Observable), Subjects for windowShown$, windowClosing$, customData$, workspaceSave$. Cleans up through DestroyRef.
- Injection tokens: HOST_RUNTIME, HOST_CONFIG_MANAGER, HOST_CONFIG_URL.

5.4 @starui/config-service-react

- <ConfigServiceProvider> — bootstraps createConfigManager({appId, identity, seedConfigUrl, configServiceRestUrl, dataServices}), runs migrateLegacyProfilesIfNeeded on first boot, builds a createConfigServiceStorage factory, publishes ApplicationContext into the surrounding <DataServicesProvider>.
- Context value: { configManager, storage, appId, userId, applicationContext }.
- useConfigService() hook.
- Disposed-guard against late init() resolutions when React Strict Mode remounts effect cleanups (race fix from the 2026-05-13 changelog).

5.5 @starui/config-service-angular

Angular twin:

- provideConfigService(opts) — EnvironmentProviders factory wired through provideAppInitializer(() => inject(ConfigServiceClient).init()) so ConfigManager.init() resolves before bootstrap completes.
- ConfigServiceClient (@Injectable({ providedIn: 'root' }), OnDestroy) — owns the live manager + pre-built createConfigServiceStorage factory. Sync applicationContext getter throws before init.
- CONFIG_SERVICE_OPTIONS token + ConfigServiceOptions interface (identity: AppIdentity, appId, optional seedUrl, optional restUrl).

5.6 @starui/data-services-react

- <DataServicesProvider> — lazy or eager hydration modes; eager mode uses React 19 use() for Suspense unwrap. Optional userId override.
- Hooks:
  - useDataServices() — escape hatch returning { client, appData, configStore }.
  - useAppDataStore() → { store, version, loaded }.

  - `useAppData(providerName)` → `{ values, loaded, get, set, setMany }`.
  - `useDataProviderConfig(providerId)` → `{ cfg, loading, error }`.
  - `useDataProvidersList({ subtype?, includeAppData? })` → `{ configs, loading, error, refresh() }`.
  - `useResolvedCfg(cfg)` — walks `{{name.key}}` templates against AppData; returns a stable resolved object.
  - `useProviderStream(providerId, cfg, listener)` → `{ status, error, refresh(extra?) }`. Buffered + onEvent modes.
  - `useProviderStats(providerId, listener)`.
- Internal `DataServicesUserIdContext` so list / save hooks pick up userId without prop-drilling.

5.7 `@starui/data-services-angular`

Full parity with the React hooks:

- `provideDataServices({ services })` — Angular Provider factory registering `DATA_SERVICES`.
- `DataServicesService` (`@Injectable({ providedIn: 'root' })`) — `client`, `appData`, `configStore`, raw `configManager`, `ready: Promise<void>`.
- Inject helpers (DI factories with `inject(DestroyRef)` cleanup): `injectAppDataStore()`, `injectAppData(name)`, `injectProviderStream(id, cfg, opts?)`, `injectProviderStats$(id)`, `injectDataProviderConfig$(id)`, `injectDataProvidersList$(opts)`, `injectResolvedCfg(cfg$)`.

5.8 `@starui/app-shell-react`

Single declarative root collapsing the boot order: outer-to-inner **DataServicesProvider → ConfigServiceProvider → HostWrapper → children**. Every provider is opt-in (pass a JSX element to enable; omit to skip). Theme application stays in the app entry to avoid FOUC.

---

6. Layer 4 — Widgets & shells

6.1 `@starui/grid-react` (`packages/react/widgets/grid-react/`) [stale-in-changelog]

> This is where the bulk of the grid customizer lives today. The changelog frequently references paths like `packages/core/src/modules/` or `packages/core-v2/src/...` or `packages/markets-grid-v2/...` — those locations no longer exist. After PR-8, every React grid module and its panel moved here.

The React customizer for `@starui/markets-grid`. Source-consumed (no dist).

6.1.1 Core hooks (`src/hooks/`)

`GridProvider` + `useGridPlatform`, `useModuleState(id)`, `useGridApi` / `useGridEvent`, `useProfileManager` (`UseProfileManagerResult` — `activeProfileId`, `profiles`, `isDirty`, `loadProfile`, `saveActiveProfile`, `discardActiveProfile`, create / clone / rename / delete / export / import), `useDirty` / `useDirtyCount`, `useGridColumns`, `useModuleDraft`, `useUndoRedo`, `useActiveThemeMode`.

6.1.2 Popout chrome

PopoutPortal, Poppable (with PoppableHandle / PoppableRenderProps / PopoutButtonProps), PortalContainerProvider / usePortalContainer / useResolvedPortalContainer. The popout clones <head> stylesheets into the new window, mirrors [data-theme] via MutationObserver, and falls back to inline mode on popup-blocker rejection.

6.1.3 Settings-panel primitives (src/ui/SettingsPanel/)

The "Cockpit terminal" UI kit. Every grid editor composes from these: DirtyDot, LedBar, GhostIcon, SubLabel, IconInput, PillToggleGroup / PillToggleBtn, PairRow, FigmaPanelSection, ItemCard, ObjectTitleRow, TitleInput, PanelChrome, TabStrip, Caps, Mono, SharpBtn, TGroup / TBtn / TDivider, Band, MetaCell, Stepper, SettingsRow, SummaryChip, CockpitList / CockpitListItem (cmdk-driven, ARIA-ready).

6.1.4 Local shadcn primitives (src/ui/shadcn/)

DS-themed copies of the shadcn primitives, separate from @starui/ui because grid-react predates the @starui/ui package and still owns its own copies (kebab-case filenames per the shadcn-CLI carve-out): Button, GhostIconButton (variant / size / reveal modes), Input, Textarea, Select (Radix-backed, since 2026-05-11), Switch, Popover family, full AlertDialog, Tooltip (forwardRef-capable, DS tokens), Separator, Label, ToggleGroup, ColorPicker / ColorPickerPopover, cn.

6.1.5 ExpressionEditor (Monaco-based, src/ui/ExpressionEditor/)

- Body-mounted overflow widget (data-gc-monaco-overflow) so suggest popovers don't drift inside transform-positioned panels.
- Popout-safe DOM context — resolves Monaco helper DOM from the host's ownerDocument.
- Popout keyboard handling: Enter / Tab / Up / Down / Space wired through Monaco's model APIs so popped editors don't drop selection or insert ghosts.
- Live onChange propagation to drafts (fixes "SUM doesn't work" UX bug where users typed but the SAVE pill stayed grey until blur).
- HelpOverlay, Palette (column + function picker), FallbackInput.

6.1.6 StyleEditor (src/ui/StyleEditor/)

One component edits the style of any AG-Grid element (cell / header / group header). Sections (opt-in via sections={[...]}): TextSection, ColorSection, BorderSection (5-row sides editor), FormatSection (FormatterPicker). CompactColorField, BorderStyleEditor.

6.1.7 FormatterPicker family (src/ui/FormatterPicker/)

- Variants: FormatterPicker, InlineFormatterPicker, CompactFormatterPicker.
- 14 value-formatter presets — Integer, 2 decimals, 4 decimals, parens-neg, red-parens-neg, green-red-nosign, green-red-$, Scientific, Basis points, plus 5 tick formats (TICK32, TICK32_PLUS, TICK64, TICK128, TICK256) for fixed-income bond prices.
- 6 currency quick-insert symbols ($, €, £, ¥, ₹, CHF). £ / ¥ / ₹ / CHF are quoted literals because SSF rejects bare glyphs.
- SSF format auto-sanitizer with try-and-quote loop.
- ISO-8601 date coercion.
- Excel color resolver: [Red] / [Green] tags produce a per-value cellStyle resolver.
- Cell-datatype auto-detection (first 20 rows sampling) — host cellDataType wins.
- ExcelReferencePopover — dark-mode-aware reference panel listing Excel token categories.

6.1.8 format-editor primitives (`src/ui/format-editor/`)

Figma-inspired set: `FormatPopover` (Radix-Popover wrapper, portal-based, collision-detected, popover-stack-aware), `FormatDropdown`, `FormatColorPicker`, `EDGE_ORDER`, `defaultSideSpec`, `makeDefaultSides`.

6.1.9 Grid modules (every registered module the platform supports)

Module id	Priority	Schema	Panel?	What it does
`general-settings`	0	v3	✓ `GridOptionsPanel` (v5 sidebar nav + summary chips)	60 grid options across 8 bands: Essentials / Row grouping / Pivot+totals / Filter+sort+clipboard / Editing+interaction / Styling / Default ColDef (7 subsections) / Performance
`column-templates`	5	—	✗ (authored from FormattingToolbar + Column Settings chip-strip)	Reusable override bundles. `pickTemplateFields`, `snapshotTemplate`, `resolveTemplates`, update/rename reducers. Type defaults per `ColumnDataType`.

Module id	Priority	Schema	Panel?	What it does
`column-customization`	10	v9	✓ `ColumnSettingsPanel` (8 bands: HEADER / LAYOUT / TEMPLATES / CELL STYLE / HEADER STYLE / VALUE FORMAT / FILTER / ROW GROUPING)	Per-column overrides. Per-theme styling wrapper (`{dark?, light?}`). Global cell formatter is split into number + date slots since 2026-05-13. Reducers for every facet, AppData hooks for `valuesSource`, transforms emit CSS scoped to `.ag-cell.ds-rule-*` and `.ag-row.ds-rule-*` (no broad `.ds-rule-*` selectors).
`calculated-columns`	15	—	✓ `CalculatedColumnsPanel`	Expression-driven virtual columns. Column-wide aggregations (SUM([price]) sums every row). `cellValueChanged` / `rowValueChanged` / `rowDataUpdated` trigger targeted refresh of virtual col ids.

Module id	Priority	Schema	Panel?	What it does
`column-groups`	18	—	✓ `ColumnGroupsPanel`	Nestable groups (depth cap 3). `openGroupIds` round-trip via `columnGroupOpened` listener. `marryChildren` / `openByDefault` toggles. Header styling via injected CSS on `gc-hdr-grp-{groupId}` with `::after` border overlay.
`conditional-styling`	20	—	✓ `ConditionalStylingPanel`	Expression-driven row/cell painting. Indicators (20+ Lucide icons + position + target). Flash with mode (`oneShot` / `pulse`), 8 palette colors, single `durationMs`, per-rule `@keyframes ds-flash-<ruleId>`. Optional `activeDurationMs` window (style reverts after match). Immediate diff-aware refs (`[col.old]` / `[col.new]`). Coalesced expiry timer, targeted refresh batcher, cross-column trigger refresh.

Module id	Priority	Schema	Panel?	What it does
`saved-filters`	1001	v2	✗ (authored from `FiltersToolbar`)	Opaque saved-filter records (`id`, `label`, `active`, `filterModel`). Validation on deserialize drops non-boolean / corrupt rows.
`toolbar-visibility`	1000	—	✗	`Record<toolbarId, boolean>` — round-trips toolbar layout across profile load / save. Today consumed by `FiltersToolbar` for collapse/expand state.
`grid-state`	200	v3	✗	Captures native AG-Grid state (column order/widths/pinning/sort/filters/gr open-closed/pagination/sidebar/focus, group expansion + viewport anchor + quickFilterText) on **explicit Save only**. Replayed on `onGridReady` and `profile:loaded`. Stale-order merge for new calc columns; selection-column position + pinning re-applied via `applyColumnState` + `firstDataRendered`.

6.1.10 Other surfaces

- **Conditional-styling rule list** — clone action per rule (full deep-copy, sits beside source, starts inactive). Per-rule delete on the list row. Dirty-state RESET pill in Conditional Styling / Column Groups / Calculated Columns editors.

6.2 `@starui/markets-grid` (packages/react/widgets/markets-grid/) [stale-in-changelog]

> The changelog also has frequent references to `packages/markets-grid-v2/` — that location is gone. The widget now lives here.

The flagship React widget. Source-consumed (no dist). Registers `AllEnterpriseModule` once, installs the AG-Grid set-filter validate guard, and wires the 9 modules from `@starui/grid-react` into `DEFAULT_MODULES` (`generalSettingsModule` … `gridStateModule`).

6.2.1 `<MarketsGrid>` component & handle

- **Imperative handle** (`MarketsGridHandle`) via `forwardRef` + `onReady`: `gridApi`, `platform`, `profiles` (`UseProfileManagerResult`), `saveAll(): Promise<void>`, plus `getConfig()` / `setConfig()` when the local-storage bundle factory is used.
- **Props (additive, all optional)**: `gridId`, `rowData`, `columnDefs`, `modules`, `theme`, `rowIdField` (string or composite-key array), `appData` (`AppDataLookup`), toolbar / visibility flags (`showToolbar`, `showFiltersToolbar`, `showFormattingToolbar`, `showSaveButton`, `showSettingsButton`, `showProfileSelector`), `sideBar`, `statusBar`, `defaultColDef`, `rowHeight`, `headerHeight`, `animateRows`, legacy `storageAdapter`, `autoSaveDebounceMs`, `instanceId`, `appId`, `userId`, `storage` factory, admin slot `adminActions`, controlled `gridLevelData` / `onGridLevelDataLoad`, `componentName`, `headerExtras` slot, `caption`, `tabsHidden`, `onCaptionChange`, `onSavingChange`, `onGridReady`.
- **Storage factory model**: `StorageAdapterFactory(opts: { instanceId, appId, userId, gridId }) => StorageAdapter`.
- **`createMarketsGridLocalStorageStorage()`** — returns a branded factory backed by `LocalStorageBundleAdapter`. The bundle key is `markets-grid-bundle:${gridId}`; the active profile id mirrors at `gc-active-profile:${gridId}` for ProfileManager. No `appId`/`userId` needed.
- **Default fallback** — when no `storage` is passed, the grid warns once in dev and uses an in-memory adapter.

6.2.2 Chrome surfaces

- **`PrimaryToolbar`** — single `.gc-primary-row` flex strip hosting: caption + FiltersToolbar slot (`flex:1`) → Brush (formatting-toolbar toggle) → divider → ProfileSelector → divider → Save → divider → Settings → divider → grid-info popover → admin-actions cluster.
- **`ProfileSelector`** — full profile menu with create / clone / rename / delete / lock / export / import per-row, plus active-profile footer. Reserved Default profile is non-deletable. Hover-revealed per-row icons via `GhostIconButton`.
- **`FiltersToolbar`** — pill row for saved filter models with edit / trash / funnel actions. Collapse / expand carousel (chevron toggle swaps for a compact `N filters · M active` chip). Sticky Clear + Add outside the scrollable section. `+` button uniqueness check spans active AND inactive pills (`isNewFilter`). New pill captures the delta filter model, not the merged model (`subtractFilterModel`).
- **`FormattingToolbar`** — poppable in-grid horizontal or floating vertical. The floating panel rides a `DraggableFloat` wrapper with grip handle, viewport clamp, window-resize re-clamp, and a built-in close button. Eight formatter "modules" inside: `ModuleContext` (column-label chip — inline rename when one column selected), `ModuleClear` (Clear-selected and Clear-all

actions), ModuleLibrary (Templates menu), ModulePaint (border / background editor), ModuleFormat (FormatterPicker + currency quick-insert + tick denominator buttons), ModuleType (Bold/Italic/Underline/Strike, alignment, size, weight), ModuleEditorFilter (cell-editor + filter-kind picker, includes the StreamSafe filter variants).
- **SettingsSheet** — settings drawer (Popover + Sheet + Poppable) with Help integration, DIRTY=NN counter via useDirtyCount, OpenFin-aware popout button (v2-settings-popout-btn).
- **HelpPanel** + help/ sections — EmojiGrid, EmojiSection, ExcelSection, ExpressionsSection, Overview, TradingSection, TrafficLightSection.
- **AdminActionButtons** — renders the adminActions cluster with lucide-icon mapping (lucide:* ref).
- **GridInfoButton** — info popover (path / instanceId / appId / userId / gridId).
- **EditableCaption** — inline pencil/Input editor surfaced when tabsHidden === true, persisted via onCaptionChange.
- **UnsavedSwitchDialog** — shadcn AlertDialog for profile-switch while dirty (Save & switch / Discard / Cancel).
- **TemplateManager** — popover-based template list with per-row Update / Rename / Delete plus the "Will save: …" caption summarising capturable categories.

6.2.3 Floating filters (streamSafe*FloatingFilter*.ts)

- **streamSafeFloatingFilter** — text, comma-token OR matching, clear button.
- **streamSafeNumberFloatingFilter** — operators (>, >=, <, <=, =), ranges (100-150), compound (>100 and <150).
- **streamSafeDateFloatingFilter** (new 2026-05-17) — ISO, MM/DD/YYYY (US), DD/MM/YYYY (EU; auto-detected when a part exceeds 12 or forced via floatingFilterComponentParams.dateLocale: 'eu'), DD.MM.YYYY (EU dot), month-name with optional ordinal, quarter (Q1 2025), Unix epoch (10-digit s / 13-digit ms), relative keywords (today / yesterday / tomorrow). Compound grammar mirrors number filter. Partial inputs auto-expand to inRange.
- Shared DOM scaffolding in streamSafeFloatingFilterDom.

6.2.4 Theme + OpenFin helpers

- **useGridTheme** (theme/) — public hook that resolves the AG-Grid Theme (Quartz tokens), following [data-theme] on <html>.
- **openfinViewProfile.ts** — createOpenFinViewProfileSource() returning an ActiveIdSource that reads / writes customData.activeProfileId for per-view active profile (workspace duplicate semantics).

6.3 @starui/widgets-react (packages/react/widgets/widgets-react/)

The "production composition" layer that wraps markets-grid for hosted contexts.

6.3.1 Blotter primitives (legacy)

BlotterToolbar, LayoutSelector, BlotterProvider + useBlotterDI, blotter types, interfaces (IBlotterDataProvider, IActionRegistry), useBlotterDataConnection, useGridStateManager. Useful for the widget-SDK-style hosts but not for MarketsGrid.

6.3.2 /hosted subpath — HostedMarketsGrid

The consolidated hosting wrapper. Single component replacing the previous six-deep stack (BlottersMarketsGrid → HostedFeatureView → HostedComponent → BlotterGrid → MarketsGridContainer → MarketsGrid).

- Composes: DataServicesProvider → FullBleed → ConfigManagerLoadingGuard → MarketsGridContainer → <MarketsGrid>.
- Owns identity resolution (OpenFin + browser-fallback), ConfigService-backed storage with auto-injected registered-component metadata, AG-Grid blotter theme, ConfigManager loading guard, document title, one-shot legacy view-state cleanup.
- Flat props (no gridProps escape hatch).
- Exports HostedContext, RegisteredComponentMetadata, ConfigManager, StorageAdapterFactory, HostedMarketsGridProps.

6.3.3 useHostedView + sub-hooks

Hook-based public API for any feature hosted inside the OpenFin shell. All degrade safely outside OpenFin.

- **useHostedIdentity** — OpenFin / browser identity resolution + storage adapter wrapping.
- **useAgGridTheme** — DS-themed AG-Grid theme for hosted grids.
- **useIab** — { subscribe, publish } over fin.InterApplicationBus, auto-cleanup on unmount.
- **useOpenFinChannel** — Channel-API factory with provider / client teardown on unmount.
- **useTabsHidden** — boolean tracking the parent OpenFin window's tab-strip visibility (via the shared options-changed listener) + deriveTabsHidden.
- **useWorkspaceSaveEvent(cb)** — connects to the platform-side Channel provider (marketsui-workspace-save-channel) and registers an awaited flush handler so the workspace snapshot includes the latest in-memory state.
- **useColorLinking** — { color, linked } from the parent window's workspace-platform color/link state + deriveColorLinking.
- **useFdc3Channel** — { current, join, leave, addContextListener, broadcast } thin wrapper over window.fdc3 user channels.
- **useHostedView(args)** — umbrella composer that calls every sub-hook plus useHostedIdentity and useAgGridTheme. Single result bag with stable identity-keyed callbacks.

6.3.4 /v2/markets-grid-container

MarketsGridContainer — two-provider container with hidden toolbar (revealed via Shift+Ctrl+P chord), live/historical mode toggle, AsOf date picker that writes into AppData (positions.asOfDate) → resolved template re-attach → Hub.restart(extra). ProviderToolbar + ProviderMode, DatePicker, MarketsGridLoadingOverlay (glassmorphism dual-ring spinner), useChordHotkey. Caption persistence through the same gridLevelData row.

6.3.5 /v2/provider-editor

DataProviderEditor — outer list + form shell; popout-ready, viewport- fit. EditorForm with 4 tabs (Connection · Fields · Columns · Behaviour) + Diagnostics when editing existing. useProviderProbe hook. Tabs: ConnectionTab, FieldsTab (windowed past 100 rows), ColumnsTab (uses useGridTheme from markets-grid via useAgGridTheme), DiagnosticsTab. Transports: StompFields, RestFields, MockFields, AppDataFields, BehaviourFields. Helpers KeyValueEditor, MultiSelect, ensureProviderEditorAgGridModules.

6.3.6 /v2/data-provider-selector

DataProviderSelector — dropdown or list mode. Reads useDataProvidersList(). onEdit / onCreate callbacks for popout launching.

6.4 `@starui/widgets-angular`

Subset of the React widgets — Angular parity is catching up. No HostedMarketsGrid yet; no MarketsGrid wrapper yet.

- **DockConfigurator** (`star-dock-configurator`): tree + properties split editor with import/export JSON and Apply button. Sub-components: `TreeNodeComponent` (recursive), `PropertiesPanelComponent`.
- **DataProviderEditor** (`star-data-provider-editor`): provider list + form layout. `ProviderListComponent`, `ProviderFormComponent`, `StompFormComponent` (3-tab STOMP editor with `probeStomp` + `inferFields` + AG-Grid Enterprise field-tree picker).
- Services: `DataProviderService` (Observable-returning CRUD over `DataServicesService.configStore`), `FieldInferenceService` (RxJS BehaviorSubject-backed state for the STOMP inference workflow).
- `StarWidgetsModule` (`NgModule`) — imports + exports the two standalone components.

---

7. Layer 5 — Tools & dev UIs

7.1 `@starui/config-browser` (`packages/react/tools/config-browser-react/`)

React admin UI for the six Dexie / REST tables (`appConfig`, `appRegistry`, `userProfile`, `roles`, `permissions`, `pendingSync`).

- `ConfigBrowserPanel` — full UI; OpenFin theme-syncs via `fin.IAB` `theme-changed`.
- Sub-components: `TableSidebar`, `Toolbar` (quick-filter, theme, import/export, refresh, deleteAll), `DataGrid` (AG-Grid with custom theme), `RowDrawer` (JSON editor), `ImportPreviewDialog` (multi-mode), `DeleteAllDialog`.
- `useConfigBrowser` hook — `hostEnv`, `restUrl`, selected table, rows, counts, isLoading, `refresh`, `saveRow`, `deleteRow`, `previewImport`, `importRows`, `deleteAllRows`, `exportAll`. Export `ImportMode` / `ImportPreview`.
- `createConfigBrowserAction({ launch })` — produces an `AdminAction` entry for the MarketsGrid Tools dropdown with default id `'config-browser'`, icon `'lucide:database'`.

7.2 `@starui/config-browser-angular` (`packages/angular/tools/config-browser-angular/`)

Angular twin. `ConfigBrowserComponent` (`mkt-config-browser`), built on PrimeNG primitives (`pInputText`, `pButton`, `pTextarea`) and `ag-grid-angular` themed via `agGridDarkParams` / `agGridLightParams`. Inline right-docked JSON editor (no PrimeNG drawer portal). `ConfigBrowserService` (`@Injectable()`, component-scoped) resolves the `ConfigManager` via the optional `ConfigServiceClient` or the platform singleton.

7.3 `@starui/config-editor-ui` (`packages/react/tools/config-editor-ui/`)

Engine-agnostic React editors for the four auth tables:

- Per-table editors: `RolesEditor`, `PermissionsEditor`, `UserProfileEditor`, `AppRegistryEditor`. Built on shared `EditorShell` + `EditorDataTable` + `OptimisticLockDialog`.
- Matrix surfaces: `PermissionMatrix`, `RoleAssignmentMatrix` (with `RoleAssignmentMode`).
- Optimistic locking: `EditorOptimisticLockError`, `guardOptimisticUpdate`, `useOptimisticUpdate`.

- Validation helpers: validateRole, validateRoleDelete, validatePermission, validatePermissionDelete, validateUserProfile, validateUserProfileDelete, validateAppRegistry; hasBlockingError, formatErrors, ValidationError, ValidationSeverity.
- createStubClient for tests.

7.4 @starui/workspace-setup-react (packages/react/tools/workspace-setup-react/)

The unified 3-pane workspace setup editor (Components catalog → Dock layout → Inspector). Supersedes the old separate Dock Editor + Registry Editor windows.

- WorkspaceSetup — reads OpenFin customData via readHostEnv() to forward (appId, userId) scope, primes platform default scope, lazy-mounts body once scope is resolved.
- Panes: ComponentsPane, DockPane, InspectorPane (form for whatever is selected). EditorSelection, newDraftEntry.
- Top-level ActionButtons + nestable DropdownButtons via tree mutations ('+ New menu, per-row+ Add' popover, X delete, up/down reorder chevrons).
- ImportConfig — default-exported popout window.
- Composition hooks: useDockEditor, useRegistryEditor. Both add non-destructive reload() so Discard reverts rather than wiping IDB.
- Icon machinery: iconIdToSvgUrl, parseIconUrl, iconIdToThemedUrls, ICON_OPTIONS, DEFAULT_ICON, findIconByName, IconOption, IconPicker component (writes iconId not display name).
- Cascade prune on registry deletion: handleDelete walks the dock tree and dispatches REMOVE_BUTTON / REMOVE_MENU_ITEM for every item referencing the deleted entry.
- .bn-scrollbar utility for themed scrollbars.

---

8. Apps

8.1 apps/demo-react — Primary E2E target

- Stack: React 19 + Vite 7. Library deps as hashed tarballs from libs/.
- <AppShell> wraps the tree with BrowserRuntime + createConfigClient + createConfigManager (Dexie); runs migrateLegacyProfilesIfNeeded once at boot; first render gated on storageReady (a createConfigServiceStorage factory promise).
- URL-driven view switcher (?view=...): single / dashboard / depth / design-system / stockflux-blotter / fixture (?f=<name>).
- Single-grid view seeds a "Showcase" profile via ensureShowcaseSeed() on first boot per gridId, then writes activeProfileKey(GRID_ID) so no profile flash on mount.
- Live ticking toggle (startLiveTicking with applyTransactionAsync, persisted via gc-ticking localStorage).
- Theme toggle via useHost().setTheme (broadcasts across windows).
- Playwright tests under e2e/ (baseline 195 / 214 passing per docs/E2E_STATUS.md).

8.2 apps/demo-angular

- Stack: Angular 21 standalone components, PrimeNG 21 themed via @starui/design-system/primeng (Aura preset). app.routes.ts is empty — the dock manager owns tab switching.
- Bootstraps with provideAnimationsAsync + providePrimeNG({ preset: ChromaDeskPreset }). The preset is definePreset(Aura, primengPreset) mapping DS tokens to PrimeNG.

- Root App owns DockManagerCoreComponent (@widgetstools/angular-dock-manager) with slateDark / vsCodeLight themes; layout serialised via serialize / deserialize / collectAllPanelsOrdered.
- Registers 32 standalone widget components (chart, orderBook, tradeTicket, blotter, rfq, bondBlotter, riskKpi, bookRisk, dv01, scenario, varTrend, riskLimits, indices, econCal, intraday, yieldCurve, orderKpi, orderBlotter, orderDetail, oasDur, durBuckets, sectors, histOas, oasDist, pnl, researchList, noteDetail, designSystem, ...).
- Pre-built layouts: Trade / Prices / Risk / Market / Orders / Research / Design tabs.
- Services: TradingDataService (+ TICKER_STRIP), SharedStateService, cell-renderers, ag-grid-theme.
- **Does NOT yet consume the @starui/*-angular packages** (only @starui/design-system). HostService / ConfigService / DataServices / widgets-angular all exist but aren't wired here.

8.3 apps/demo-configservice-react

- Stack: React 19. Consumes @starui/* via workspace symlinks (no tarballs).
- Single-bundle multi-surface entry: renders <App /> normally or <ConfigBrowserPopout /> when URL has ?configBrowser=1 (opened via window.open from the main grid toolbar). Both wrapped in <AppShell> with BrowserRuntime.
- createConfigClient({ baseUrl: import.meta.env.VITE_CONFIG_SERVICE_URL }) — env-driven Dexie-vs-REST switch.
- **Multi-user demo** — header User picker between dev1, alice, bob. Active user persists via demo-cs-current-user localStorage; user-scoped active-profile key scopedActiveProfileKey so per-user profile-set isolation is visible.
- Views: single, dashboard, depth.
- MarketsGrid adminActions includes createConfigBrowserAction for the popout.
- Publishes the ConfigManager to @starui/openfin-platform/config (setConfigManager) so the popout resolves the same instance; host env encoded via encodeHostEnvForQueryString.

8.4 apps/markets-ui-react-reference

The reference OpenFin shell.

- Stack: React 19 + react-router v6, OpenFin (@openfin/core 43.101.2, @openfin/workspace 23.0.20, @openfin/workspace-platform, @openfin/notifications), @finos/fdc3, @tanstack/react-query, AG-Grid 35.1, shadcn primitives.
- Runtime selection: OpenFinRuntime.create() when isOpenFin(), else BrowserRuntime. appId = "markets-ui-react-reference", userId = "dev1". REST endpoint resolved via getConfigServiceRestUrlFromManifest() from manifest.fin.json.
- Routes:
  - /platform/provider — hidden bootstrap window. Runs initWorkspace() from @starui/openfin-platform. Background prefetch of every tool-window route chunk.
  - / — landing.
  - /views/view1, /views/view2 — sample views launched as OpenFin Views.
  - /blotters/marketsgrid — BlottersMarketsGrid, delegates to <HostedMarketsGrid> with dataServices (mainThread bundle), eager hydration, historicalDateAppDataRef: 'positions.asOfDate', and an onEditProvider opening openProviderEditorPopout(runtime, { providerId }).
  - /dataproviders — <DataProviderEditor> inside <DataServicesProvider>. Reads ?id= for direct selection.
  - /config-browser — ConfigBrowserPanel from @starui/config-browser.
  - /import-config, /workspace-setup — lazy-loaded ImportConfig and WorkspaceSetup from @starui/workspace-setup-react.

  - `/rename-view-tab` — popout for OpenFin tabstrip rename; sets `document.title` on the target view via `executeJavaScript`.
- Provider stack on every route (except `/platform/provider`): `<ViewRoutesLayout>` → `<AppShell>` → `<DataServicesProvider>` → `<ConfigServiceProvider>` → `<HostWrapperWithProviderClient>` (derives a `ConfigClient` off `useConfigService().configManager`).
- `dataServices.mainThread.ts` — per-app data-services bundle via `createDataServicesClient({ appName: 'TestApp', userId: LOGGED_IN_USER_ID, configServiceRestUrl })`.

8.5 apps/demo-apps/basic-starui-app

The minimal "just use MarketsGrid in your own app" example.

- Stack: React 19. Library deps as tarballs from `../../libs/`.
- Mounts `<MarketsGrid>` against `createMarketsGridLocalStorageStorage()` — pure localStorage adapter. No ConfigService, no AppShell, no runtime port.
- `GRID_ID = 'bond-blotter-v1'`. Seeded bond inventory (`buildBondInventory(180)`) with `bondColumnDefs` / `bondDefaultColDef`.
- Theme handled directly via `applyTheme` / `getTheme` from `@starui/design-system`.
- **Bundle-adapter pattern** — uses `marketsGridLocalStorageBundleKey(GRID_ID)` and `activeProfileKey(GRID_ID)` from `@starui/core` to read the same localStorage bundle the grid persists to. That's how the chrome reflects layout changes without an adapter.
- **StatusStrip** (`src/components/StatusStrip.tsx`) — bottom strip showing rows / notional / dv01 / day-pnl / IG / HY counts derived from the live rows array; wired to `platform.events.on('profile:loaded' | 'profile:saved')` so it reflects only real changes (no background polling).
- **ConfigInspector** (`src/components/ConfigInspector.tsx`) — shadcn Sheet-based right drawer with Tabs that reads `marketsGridLocalStorageBundleKey` to surface the raw `MarketsGridLocalStorageConfig` JSON. Manual Refresh button (no interval poll). Copy / Clear / Active-indicator actions.
- AppMenubar — shadcn `Menubar` with File (reset/export/import) + View (theme toggle, open inspector) menus.
- Three seeded layouts mentioned in changelog (`trader-console`, `risk-and-pnl`, `relative-value`) live in MarketsGrid's built-in showcase seeding, not as app-level seeds.

8.6 apps/stomp-view-server

- Stack: Node ≥ 18 + TypeScript, `ws` 8.16, `dotenv`. A **server**, not a view.
- Default endpoint `ws://localhost:8081` (overridable via `PORT`).
- Serves synthetic fixed-income snapshots (default 20k rows; clampable via STOMP send header `snapshot-rows` / `row-count`; min 1k / max 20k)
  - live updates.
- HTTP routes: `/health` (status / uptime / memory / snapshot config), `/` (service metadata + protocol description).
- Per-client `StompConnection` tracked in `Map<number, StompConnection>`.
- Data generation: `data/fiRecords.ts`, `data/mutate.ts`, `data/rng.ts`, `util/hash.ts`.
- Protocol contract: `protocol/contract.ts` (matches `stomp-fixed-income-server`).
- Config knobs (`src/config.ts`): `PORT`, `NODE_ENV`, `DEFAULT/MIN/MAX_SNAPSHOT_ROWS`, `DEBUG`, `LOG_OUTBOUND`, `LOG_LIVE_EVERY`, `LOG_BODY_PREVIEW`.

---

9. Cross-cutting features

9.1 Design-system tokens (`--ds-*`)

Every UI surface in the monorepo resolves through `--ds-*` CSS variables. The legacy families (`--bn-*`, `--fi-*`, `--gc-*`, `--ck-*`, `--mdl-*`) have been swept out by the 2026-05-09 unification. The mapping (partial):

Legacy	Unified
`--bn-bg` / `--fi-bg1`	`--ds-surface-primary`
`--bn-bg2` / `--fi-bg2`	`--ds-surface-secondary`
`--fi-bg0`	`--ds-surface-ground`
`--bn-t0` / `--fi-t0`	`--ds-text-primary`
`--bn-border` / `--fi-border`	`--ds-border-primary`
`--bn-green` / `--fi-green`	`--ds-accent-positive`
`--bn-red` / `--fi-red`	`--ds-accent-negative`
`--bn-blue` / `--fi-blue`	`--ds-accent-info`
`--fi-amber`	`--ds-accent-warning`
`--fi-sans`	`--ds-font-sans`
`--fi-mono`	`--ds-font-mono`

CI gates the contract via `npm run check-ds` (`tools/scripts/check-ds-tokens.ts`, `tools/scripts/check-design-system-deps.ts`, `tools/scripts/check-react-apps-no-native-select.ts`).

9.2 Profile contract

- **Explicit Save** — `MarketsGrid` ships with `disableAutoSave: true`. The Save button is the sole write path. Module-state mutations flip an internal `isDirty` flag.
- **Dirty indicator** on the Save button (`<DirtyDot/>`).
- **Switch-while-dirty AlertDialog** — Save & switch / Discard / Cancel with testids `profile-switch-{confirm,save,discard,cancel}`.
- **`beforeunload` warning** registered only while `isDirty === true`.
- **New profile starts blank** — `resetAll()` runs before snapshotting in `createProfile`; `grid-state` clears `api.setState({}) + quickFilterText` when loaded profile has no saved state.
- **Delete safety** — cancels pending auto-save, falls back to Default with `skipFlush: true`.
- **Export / Import as JSON** — per-row Export button, footer Export (active) + Import. Import is additive (unique id + name on collision).
- **Inline rename in the popover** (`profile-rename-{id}`); Default profile is non-deletable / non-renameable.
- **Auto-save still available** when `disableAutoSave: false` (used by tests).
- **OpenFin per-view active profile** — `createOpenFinViewProfileSource()` in markets-grid reads/writes `customData.activeProfileId`. Workspace round-trip is automatic; duplicates inherit then diverge. Read priority: OpenFin override → localStorage → reserved Default.

9.3 Single-user pin (`userId = 'dev1'`)

The platform is single-user (no SSO yet). Every client-side `userId` resolution path hard-pins `LOGGED_IN_USER_ID = 'dev1'` from `@starui/runtime-port`. Touchpoints:

- `runtime-browser/resolveBrowserIdentity` — URL `?userId=` ignored.
- `runtime-openfin/resolveOpenFinIdentity` — `customData.userId` ignored.
- `openfin-platform/registryHostEnv.readHostEnv` — all three resolution paths collapse onto `DEFAULT_USER_ID` (same value as `LOGGED_IN_USER_ID`).

- widgets-react/hosted/useHostedIdentity — defaultUserId arg preserved on the public API but ignored at runtime.
- component-host/save-config.ts — build-fresh fallback uses LOGGED_IN_USER_ID instead of ''.
- data-services-react/v2.useAppData().setMany() falls back to LOGGED_IN_USER_ID for user-owner field on freshly-created AppData rows.

Replace the literal in runtime-port/types.ts (and the matching DEFAULT_USER_ID in registry-host-env.ts) when SSO lands.

9.4 Per-theme column styling

cellStyleOverrides / headerStyleOverrides store values under a theme-keyed wrapper { dark?, light? }. Helpers in packages/shared/core/src/colDef/themedStyle.ts: getActiveTheme, resolveActiveStyle, patchActiveStyle, mergeThemedStyle, migrateThemedStyle. Transform emits CSS for both slots scoped by html[data-theme="dark"] / html[data-theme="light"], so flipping themes is a pure cascade event.

9.5 OpenFin tab rename ("Save Tab As…")

Right-click → "Save Tab As..." on a view tab opens a frameless popout seeded with the current title. On confirm:

1. executeJavaScript writes document.title = "..." on the target view for the immediate tabstrip update.
2. view.updateOptions({ customData: { ..., savedTitle } }) persists the rename through the workspace snapshot.
3. On the next workspace load, OpenFinRuntime.applySavedViewTitle() reapplies customData.savedTitle to document.title, with a 3-second MutationObserver guard that defeats the page's mount-time title override.

9.6 ConfigManager init-vs-dispose race

ConfigManager.init() no longer sets isInitialized before async work finishes. A disposed flag plus single-flight initInFlight lets dispose() close Dexie while init() is pending without surfacing Dexie DatabaseClosedError to the console. Aborted inits resolve quietly; publishApplicationContext returns after appData.ready() if the manager was torn down; dispose() is idempotent.

9.7 Conditional-styling rule CSS scope

Rule CSS emits cell-scope styles against .ag-cell.ds-rule-* and row-scope styles against .ag-row.ds-rule-* .ag-cell instead of broad .ds-rule-* selectors. Indicator badges keep their own cell/header pseudo-element selectors; .ag-floating-filter is explicitly excluded so indicators don't double-paint on the filter row.

---

10. Module priorities (canonical)

```
general-settings (0)
column-templates (5)
column-customization (10)
calculated-columns (15)
```

```
column-groups (18)
conditional-styling (20)
grid-state (200)
toolbar-visibility (1000)
saved-filters (1001)
```

(The README still shows `saved-filters (30)` / `toolbar-visibility (40)` but the actual module exports in `packages/react/widgets/grid-react/src/modules/{saved-filters,toolbar-visibility}/index.ts` use 1001 and 1000 respectively. The architecture diagram and README predate the renumber.)

---

11. Build, test, and tooling

- **Package manager**: npm 10 workspaces. The root `CLAUDE.md` policy is *no `--legacy-peer-deps`*; the README still mentions the flag (stale).
- **Build orchestrator**: Turborepo 2. Every library uses `"build": "rimraf dist && tsc"` (or `ng-packagr` for Angular libs).
- **Unit tests**: Vitest 4 + jsdom 29 (baseline ~653 passing per changelog; current count drifts as packages land).
- **E2E**: Playwright 1.59 against `apps/demo-react` (baseline 195/214 passing per `docs/E2E_STATUS.md`).
- **Version pinning**: stable lines per major — React 19.2.x, Angular 21.1.x, `@openfin/core` 43.101.x. Root `overrides` keeps `@openfin/core` pinned to `43.101.4` (drop only by aligning the four workspace-platform peer packages on the same version).
- **Linting / contract gates**:
  - `npm run check-ds` — DS tokens / no hardcoded hex / no native React `<select>`.
  - `tools/scripts/check-design-system-deps.ts` — DS dep contract.
- **Propagation**: `npm run propagate -- <pkg> [...]` repacks libraries into hashed tarballs in `libs/`, rewrites every matching `file:` dep in `apps/*/package.json`, clears `node_modules/@starui/*` + `node_modules/.vite`, and runs `npm install` per app. Manifest at `libs/manifest.json` stores `{ filename, version, sha, packedAt }` per entry.

---

12. Editor coverage matrix (current)

Module / Editor	Owner package	Panel?	Pure-logic tests	E2E
`general-settings` — Grid Options	`grid-react`	✓ `GridOptionsPanel.tsx` (v5 sidebar nav)	—	✓ `v2-general-settings.spec.ts`
`column-customization` — Column Settings	`grid-react`	✓ `ColumnSettingsPanel.tsx` (8 bands)	✓ `formattingActions.test.ts`	✓ `v2-column-customization.spec.ts`
`calculated-columns`	`grid-react`	✓ `CalculatedColumnsPanel.tsx`	—	✓ `v2-calculated-columns.spec.ts`
`column-groups`	`grid-react`	✓ `ColumnGroupsPanel.tsx`	✓ `treeOps.test.ts`	✓ `v2-column-groups.spec.ts`

Module / Editor	Owner package	Panel?	Pure-logic tests	E2E
column-templates	grid-react	✗ (authored via FormattingToolbar + Column Settings chips)	✓ snapshotTemplate.test.ts	✓ v2-column-templates.spec.ts
conditional-styling	grid-react	✓ ConditionalStyling-Panel.tsx	—	✓ v2-conditional-styling.spec.ts
saved-filters	grid-react + markets-grid toolbar	✗ (FiltersToolbar IS the editor)	✓ filterSToolbar-Logic.test.ts	✓ v2-filters-toolbar.spec.ts
toolbar-visibility	grid-react	✗	—	✗
grid-state	grid-react	✗	—	● indirectly via autosave spec
Formatting Toolbar (chrome)	markets-grid	—	✓ formatter presets	✓ v2-formatting-toolbar.spec.ts

---

13. Known gaps

1. **Angular widget parity** — widgets-angular ships DockConfigurator
   - DataProviderEditor only. There is no Angular MarketsGrid wrapper, no Angular HostedMarketsGrid, no Angular RegistryEditor / ImportConfig / WorkspaceSetup yet. apps/demo-angular consumes only @starui/design-system from the workspace today.
2. **Toolbar Visibility wiring** — module state ships in every profile but concrete toolbar-toggle bindings (Brush pill, Filters toolbar show/hide) aren't routed through it. Today only FiltersToolbar collapse/expand uses it.
3. **Column Templates standalone panel** — templates are authored indirectly (save-from-toolbar, remove-via-Column-Settings-chip). A dedicated panel with rename / description / duplicate would be additive.
4. **Calculated columns in main grid header** — virtual columns appear correctly in AG-Grid's filter tool panel but a known bug shows them missing in some demo configurations.
5. **packages/shared/services/starui-mcp/** — directory exists with empty src/{handlers,tools}/ and bin/, no package.json. Looks like an in-progress MCP server scaffold; not currently wired into npm workspaces and ships no features.

---

14. Reconciliation roll-up — what IMPLEMENTED_FEATURES.md says vs reality

Items here describe features the changelog still discusses in stale terms. The *behaviour* still ships; only the *path* or *naming* moved.

Topic	Changelog phrasing	Where it actually lives today
Grid modules	`packages/core/src/modules/` or `packages/core-v2/...`	`packages/react/widgets/grid-react/src/modules/{general-settings,column-templates,column-customization,calculated-columns,column-groups,conditional-styling,saved-filters,toolbar-visibility,grid-state}/`
Settings-panel primitives	`packages/core/src/ui/SettingsPanel/` (and `core-v2/src/ui/SettingsPanel/`)	`packages/react/widgets/grid-react/src/ui/SettingsPanel/`
Format Editor primitives	`packages/core/src/ui/format-editor/`	`packages/react/widgets/grid-react/src/ui/format-editor/`
StyleEditor	`packages/core-v2/src/ui/StyleEditor/`	`packages/react/widgets/grid-react/src/ui/StyleEditor/`
FormatterPicker	`packages/core-v2/src/ui/FormatterPicker/`	`packages/react/widgets/grid-react/src/ui/FormatterPicker/`
ExpressionEditor	`packages/core/src/ui/ExpressionEditor/`	`packages/react/widgets/grid-react/src/ui/ExpressionEditor/`
`MarketsGrid` widget root	`packages/markets-grid-v2/src/MarketsGrid.tsx`	`packages/react/widgets/markets-grid/src/MarketsGrid.tsx`
`DraggableFloat`	`packages/markets-grid-v2/src/DraggableFloat.tsx`	`packages/react/widgets/markets-grid/src/DraggableFloat.tsx`
Bundle / Local-storage adapter	`LocalStorageBundleAdapter` referenced under `@grid-customizer/core` and a separate `BundleAdapter`	`packages/shared/core/src/persistence/` only — there is no separate `BundleAdapter` file
Dexie / ConfigService storage adapters	Referenced as adapters under core	They live in `@starui/config-service` (`createConfigServiceStorage` factory); core no longer ships either
`@starui/data-plane` / `@starui/data-plane-react`	Whole sub-tree of plan documents (Week 1–4, v2 rewrite, etc.)	Folded into `@starui/data-services` (vanilla) and `@starui/data-services-react` (hooks); subpath structure preserved (`./runtime`, `./client`, `./worker`) but the package names changed
Design-system tokens	Mostly `--bn-*` / `--fi-*` in older sections; later sections introduce `--ds-*`	Codebase is fully on `--ds-*`; legacy tokens are gone (lint enforces)
Module priorities	README still lists `saved-filters (30)` / `toolbar-visibility (40)`	Live exports use 1001 (saved-filters) and 1000 (toolbar-visibility)

Topic	Changelog phrasing	Where it actually lives today
`apps/markets-ui-angular-reference`	Referenced in older changelog entries and the README's "6 apps" table	Removed during consolidation. Today's app list: `demo-react`, `demo-angular`, `demo-configservice-react`, `markets-ui-react-reference`, `demo-apps/basic-starui-app`, `stomp-view-server`.
`apps/fi-trading-reference` / `apps/fi-trading-reference-angular`	Listed in README	Not present in working tree
`--legacy-peer-deps` install flag	README "Getting started" requires it	`CLAUDE.md` policy is *no* flag — plain `npm ci` should resolve cleanly; treat ERESOLVE as a real bug
Per-cell `dirtyRegistry` + `window.dispatchEvent('gc-dirty-change')`	Older phase-3 notes describe this	Every panel migrated to per-platform `DirtyBus` via `useDirty(key)`
`useDraftModuleItem` / `useModuleState(store, id)` compat shims	Many older sections reference these	Deleted; replaced by `useModuleDraft` + 1-arg `useModuleState(id)`
`BlottersMarketsGrid` / `HostedFeatureView` / `HostedComponent` / `SimpleBlotter` / `BlotterGrid` chain	Old six-deep stack referenced through several sections	Collapsed into `<HostedMarketsGrid>` in `@starui/widgets-react/hosted`
`HelpPanel` location	Sometimes referenced under `markets-grid-v2`	`packages/react/widgets/markets-grid/src/HelpPanel.tsx` and `packages/react/widgets/markets-grid/src/help/`
Module-state count	Older summary says "9 modules"	Still 9 (`general-settings`, `column-templates`, `column-customization`, `calculated-columns`, `column-groups`, `conditional-styling`, `saved-filters`, `toolbar-visibility`, `grid-state`) — accurate
Built-in expression functions	Older summary says "65+ functions"	Actual count today is 44 across 6 categories (see §4.1). The "65+" figure pre-dates the function consolidation.
Currency quick-insert symbols	"6 symbols"	Accurate ($, €, £, ¥, ₹, CHF)
Tick formats	"5 denominations (32, 32+, 64, 128, 256)"	Accurate
Value formatter presets	"14"	Accurate

Topic	Changelog phrasing	Where it actually lives today
Grid Options controls	“60 across 8 bands”	Accurate (Essentials / Row grouping / Pivot+totals / Filter+sort+clipboard / Editing+interaction / Styling / Default ColDef [7 subsections] / Performance)

---

15. How to use this document

- Start at §1 for orientation, then jump to the package you’re working in. Every section is named after its source path.
- When adding or removing a feature, update this file in the same commit (same policy as `IMPLEMENTED_FEATURES.md`, but keep that file as historical record only — feature inventory lives here).
- When the changelog and this file disagree, **this file is canonical**. Add a row to §14 (rollup) explaining the discrepancy so the next reader doesn’t have to rediscover it.

Generated 2026-05-17 by a code-first scan of `/Users/develop/wfh/starui/`. Every feature claim was verified against a source file under `packages/` or `apps/` on that date.